

FRONTEND IMPLEMENTATION PLAN

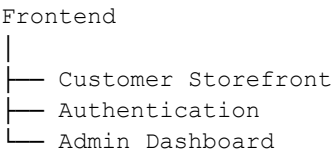
Clothing E-Commerce Platform — Core Version

1. Frontend Goal

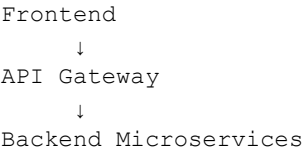
Build a complete responsive clothing e-commerce frontend with all the **standard features expected in a normal shopping website**.

The design can take visual inspiration from The Souled Store, but the project will use its **own branding, content, product data, and UI implementation**.

The frontend consists of:



The frontend communicates with the backend only through the **API Gateway**.



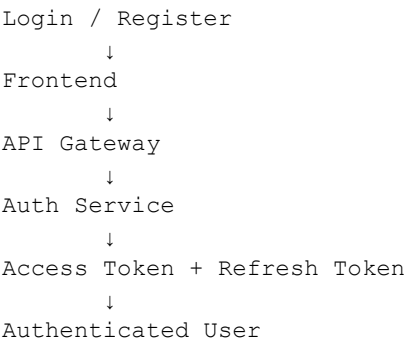
The frontend will **not directly communicate with PostgreSQL, Prisma, RabbitMQ, Redis, or individual databases**.

2. Frontend Technology Stack

Purpose	Technology
Framework	Next.js
Language	TypeScript
UI	React
Styling	Tailwind CSS
UI Components	shadcn/ui
Icons	Lucide React
API Data	TanStack Query
Client State	Zustand
Forms	React Hook Form
Validation	Zod
HTTP Client	Axios
Authentication	JWT
Product Images	Cloudinary
Payment	Stripe Test Mode
Charts	Recharts
Toast Messages	Sonner
Images	Next.js Image

Authentication

Use **JWT authentication instead of Clerk** because authentication is part of the project's backend architecture.



Roles:

CUSTOMER
ADMIN

Admin routes must only be accessible to users with the `ADMIN` role.

3. Frontend Pages

The frontend will contain **13 customer/auth feature pages + 7 admin pages**.

Customer + Authentication

#	Page	Route
1	Home	`/`
2	Products	`/products`
3	Category Products	`/category/[slug]`
4	Search	`/search`
5	Product Details	`/products/[slug]`
6	Login	`/login`
7	Register	`/register`
8	Wishlist	`/wishlist`
9	Cart	`/cart`
10	Checkout	`/checkout`
11	Order Success	`/order-success/[id]`
12	My Orders / Order Details	`/orders`, `/orders/[id]`
13	Profile	`/profile`

Admin

#	Page	Route
1	Dashboard	`/admin`
2	Products	`/admin/products`
3	Categories	`/admin/categories`
4	Inventory	`/admin/inventory`
5	Orders	`/admin/orders`
6	Payments / Refunds	`/admin/payments`

7	Analytics	`/admin/analytics`
---	-----------	--------------------

Product creation/editing uses:

```
/admin/products/new  
/admin/products/[id]/edit
```

4. Customer Navigation

Main navigation:

```
LOGO  
  
Men  
Women  
Categories  
  
Search  
  
Wishlist  
Account  
Cart
```

Basic shopping flow:

```
Home  
↓  
Products / Category  
↓  
Search & Filter  
↓  
Product Details  
↓  
Select Size / Color  
↓  
Add to Cart  
↓  
Cart  
↓  
Login / Register  
↓  
Checkout  
↓  
Address  
↓  
Payment  
↓  
Order Success  
↓  
My Orders  
↓  
Order Tracking
```

5. Home Page

Route:

/

The Home page contains:

- Navbar
- Hero banner
- Shop by category
- New arrivals
- Men's products
- Women's products
- Best sellers
- Promotional banner
- Footer

No personalized recommendation system is required.

Product Card

Each product card displays:

Product Image
Product Name
Price
Discount Price (if applicable)
Rating
Wishlist Button

Clicking a product opens its Product Details page.

6. Products Page

Route:

/products

Display:

- Product grid
- Product count
- Basic filters
- Basic sorting
- Pagination or Load More

Filters

Keep filters simple:

Category
Size
Price
Availability

Optional:

Color

Sorting

Newest
Price: Low → High
Price: High → Low

No advanced recommendation or AI search functionality is required.

7. Category Page

Route:

/category/[slug]

Examples:

/category/t-shirts
/category/hoodies
/category/jeans

Contains:

- Category title
- Product grid
- Basic filters
- Sorting

Categories may include:

T-Shirts
Shirts
Hoodies
Jeans
Joggers
Jackets
Accessories

8. Search

Route:

```
/search?q=hoodie
```

Users can search products by:

```
Product Name  
Category  
Basic Keywords
```

Search results display using the normal Product Card.

Also support:

```
No Products Found
```

No AI-based or advanced search recommendation system is required.

9. Product Details

Route:

```
/products/[slug]
```

Product Images

Display:

- Main product image
- Additional product images
- Thumbnail selection

Images are delivered through **Cloudinary**.

Product Information

```
Product Name
```

```
Rating
```

```
Price  
Original Price  
Discount
```

```
Available Color
```

```
Available Size
```

```
Stock Status
```

```
Quantity
```

```
ADD TO CART
```

ADD TO WISHLIST

Below:

- Description
- Product details
- Basic delivery information
- Reviews

No advanced recommendation engine or personalized similar-products system is required.

Stock

Unavailable sizes must be disabled.

Example:

S	M	L	XL
✓	✗	✓	✓

The stock information comes from the backend Inventory Service.

10. Authentication

Login

Route:

/login

Fields:

Email
Password

LOGIN

Also provide:

Forgot Password
Create Account

Register

Route:

/register

Fields:

First Name
Last Name
Email

Phone
Password
Confirm Password

After authentication:

Customer
↓
Store / Previous Page

Admin
↓
Admin Dashboard

JWT

The frontend uses the authentication APIs provided by the backend.

Prefer:

Short-lived Access Token
+
Refresh Token in secure HttpOnly cookie

The frontend must never contain the JWT secret.

11. Wishlist

Route:

/wishlist

Basic functionality:

- View saved products
- Remove product
- Move/Add product to cart
- Open Product Details
- Empty wishlist state

12. Cart

Route:

/cart

Each cart item displays:

Product Image
Product Name

Size
Color
Price
Quantity
Remove

Customer can:

Increase Quantity
Decrease Quantity
Remove Product

Order Summary

Subtotal
Discount
Shipping
Total

CTA:

PROCEED TO CHECKOUT

The frontend should prevent invalid quantities and show when an item becomes unavailable.

13. Checkout

Route:

/checkout

Basic checkout flow:

Delivery Address
↓
Order Review
↓
Payment

Delivery Address

Customer can:

- Select saved address
- Add address
- Edit address

Address fields:

Full Name
Phone
Address
City
State

PIN Code
Country

Order Review

Display:

Products
Size / Color
Quantity
Price

Delivery Address

Subtotal
Shipping
Discount
Total

14. Payment

Payment is part of Checkout.

Use **Stripe Test Mode**.

```
Checkout
  ↓
Frontend requests payment
  ↓
Backend Payment Service
  ↓
Stripe
  ↓
Payment submitted
  ↓
Backend verifies payment
  ↓
Order confirmed
```

Frontend handles:

- Payment UI
- Loading state
- Success state
- Failed state

Backend handles actual payment verification and Stripe webhooks.

The frontend must never contain the Stripe secret key.

15. Order Success

Route:

```
/order-success/[id]
```

Display:

```
Order Successfully Placed
```

```
Order ID
```

```
Products
```

```
Total Amount
```

```
Delivery Address
```

```
Payment Status
```

```
TRACK ORDER
```

```
CONTINUE SHOPPING
```

16. My Orders & Order Tracking

Routes:

```
/orders
```

```
/orders/[id]
```

My Orders

Display:

```
Order ID
```

```
Date
```

```
Amount
```

```
Status
```

```
Products
```

```
View Details
```

Order Tracking

Keep tracking simple:

```
Order Placed
```

```
↓
```

```
Confirmed
```

```
↓
```

```
Packed
```

```
↓
```

```
Shipped
```

```
↓
```

```
Delivered
```

Also support:

Cancelled
Refunded

Customer can:

- View order
- Track order
- Cancel eligible order
- Request refund where allowed

17. Customer Profile

Route:

/profile

Sections:

Personal Information
Addresses
My Orders
Wishlist
Password / Security
Logout

Customer can:

- Edit basic profile
- Manage addresses
- Change password
- Logout

Profile photo upload is optional and not required for the core version.

18. Admin Layout

Admin has a separate dashboard layout.

Admin

- Dashboard
- Products
- Categories
- Inventory
- Orders
- Payments
- Analytics
- Logout

Admin pages use:

- Sidebar
- Topbar
- Content area

19. Admin Dashboard

Route:

```
/admin
```

Display basic summary cards:

```
Total Products
Total Orders
Total Revenue
Low Stock Products
```

Also display:

- Recent orders
- Low-stock products
- Basic revenue chart

No advanced business intelligence dashboard is required.

20. Product Management

Route:

```
/admin/products
```

Display:

```
Image
Product Name
Category
Price
Stock
Status
Actions
```

Actions:

```
Add
Edit
Delete
View
```

Add Product

Route:

```
/admin/products/new
```

Basic fields:

```
Product Name
Description
Category
Price
Discount (optional)
Sizes
Colors
Images
Status
```

Edit Product

```
/admin/products/[id]/edit
```

Admin can update existing product information.

21. Cloudinary Image Upload

Cloudinary will be used for **product image storage**.

```
Admin
↓
Add/Edit Product
↓
Select Images
↓
Upload
↓
Cloudinary
↓
Image URL
↓
Backend stores URL
```

Frontend UI needs:

```
Select Images
Preview Images
Upload
Remove Image
Upload Progress
Upload Error
```

Multiple product images should be supported.

Cloudinary credentials/secrets must not be exposed in frontend code.

The backend team will provide the secure upload/signature mechanism.

22. Categories Management

Route:

```
/admin/categories
```

Admin can:

```
View Categories
Add Category
Edit Category
Delete Category
```

Keep category management basic.

23. Inventory Management

Route:

```
/admin/inventory
```

Display:

```
Product
Size
Color
Available Stock
Status
```

Admin can:

- View stock
- Update stock
- Search product
- View low-stock products

No multi-warehouse management is required.

24. Admin Order Management

Route:

```
/admin/orders
```

Display:

Order ID
Customer
Amount
Payment Status
Order Status
Date

Admin can update:

Confirmed
Packed
Shipped
Delivered
Cancelled

25. Payments & Refunds

Route:

/admin/payments

Display:

Payment ID
Order ID
Customer
Amount
Status
Date

Statuses:

Successful
Failed
Refunded

Admin can:

- View payment
- View failed payments
- View refund status
- Process/approve refund where backend rules permit

26. Basic Analytics

Route:

/admin/analytics

Keep analytics simple.

Use Recharts for:


```
Revenue Chart
Orders Chart
```

Basic information:

```
Total Revenue
Total Orders
Total Products
```

No advanced forecasting, customer segmentation, or complex business intelligence is required.

27. Frontend State Management

Use **TanStack Query** for backend/server data:

```
Products
Profile
Orders
Inventory
Payments
Analytics
```

Use **Zustand** for lightweight frontend state:

```
Cart UI
Wishlist UI
Authentication State
Filters
UI State
```

Persistent backend data remains controlled by the backend.

28. API Layer

Keep all API communication centralized.

```
services/
├── api.ts
├── auth.service.ts
├── user.service.ts
├── product.service.ts
├── cart.service.ts
├── order.service.ts
├── inventory.service.ts
├── payment.service.ts
└── admin.service.ts
```

Components should **not directly contain scattered Axios requests**.

Example:

Product Page
↓
product.service.ts
↓
API Gateway
↓
Backend

29. Backend Requirements Derived From Frontend

This section is especially important because the **backend team will create its implementation plan based on this frontend plan.**

Frontend Feature	Backend Capability Required
Register/Login	Authentication
JWT Session	Auth APIs
Profile	User management
Addresses	Address CRUD
Products	Product catalog
Categories	Category management
Search	Product search
Filters	Product filtering
Product Details	Product + variants
Wishlist	Wishlist CRUD
Cart	Cart CRUD
Stock	Inventory availability
Checkout	Order creation
Payment	Stripe integration
My Orders	Order retrieval
Tracking	Order status
Cancellation	Order cancellation
Refund	Refund processing
Admin Products	Product CRUD
Product Images	Cloudinary support
Admin Categories	Category CRUD
Admin Inventory	Stock management
Admin Orders	Order management
Admin Payments	Payment management
Dashboard	Summary data
Analytics	Revenue/order data

30. Loading, Empty & Error States

Important pages must support:

```
Loading
Success
Empty
Error
```

Examples:

```
Loading Products
No Products Found
Empty Cart
Empty Wishlist
No Orders
Payment Failed
Product Out of Stock
Network Error
Unauthorized
404 Page
```

31. Responsive Design

Frontend must work on:

```
Mobile
Tablet
Laptop
Desktop
```

Important responsive behavior:

- Mobile navigation
- Responsive product grid
- Mobile filter drawer
- Responsive cart
- Responsive checkout
- Responsive admin tables

32. Final Frontend Folder Structure

```
frontend/
|
├── app/
|   |
|   ├── (store)/
|   |   └── page.tsx
```

```
— products/
  |— page.tsx
  |— [slug]/
    |— page.tsx

— category/
  |— [slug]/
    |— page.tsx

— search/
  |— page.tsx

— wishlist/
  |— page.tsx

— cart/
  |— page.tsx

— checkout/
  |— page.tsx

— order-success/
  |— [id]/
    |— page.tsx

— orders/
  |— page.tsx
  |— [id]/
    |— page.tsx

— profile/
  |— page.tsx

— (auth)/
  |— login/
    |— page.tsx
  |— register/
    |— page.tsx

— admin/
  |— layout.tsx
  |— page.tsx
  |— products/
    |— page.tsx
    |— new/
      |— page.tsx
    |— [id]/
      |— edit/
        |— page.tsx

— categories/
  |— page.tsx

— inventory/
  |— page.tsx

— orders/
  |— page.tsx

— payments/
```

```

├── analytics/
│   ├── page.tsx
│   └── page.tsx
├── layout.tsx
├── loading.tsx
├── error.tsx
├── not-found.tsx
├── components/
│   ├── ui/
│   ├── layout/
│   │   ├── Navbar.tsx
│   │   ├── Footer.tsx
│   │   └── AdminSidebar.tsx
│   ├── home/
│   ├── product/
│   ├── search/
│   ├── auth/
│   ├── wishlist/
│   ├── cart/
│   ├── checkout/
│   ├── order/
│   ├── profile/
│   ├── admin/
│   └── charts/
├── services/
│   ├── api.ts
│   ├── auth.service.ts
│   ├── user.service.ts
│   ├── product.service.ts
│   ├── cart.service.ts
│   ├── order.service.ts
│   ├── inventory.service.ts
│   ├── payment.service.ts
│   └── admin.service.ts
├── hooks/
│   ├── useAuth.ts
│   ├── useCart.ts
│   ├── useWishlist.ts
│   ├── useProducts.ts
│   └── useOrders.ts
├── store/
│   ├── auth.store.ts
│   ├── cart.store.ts
│   ├── wishlist.store.ts
│   └── ui.store.ts
├── schemas/
│   ├── auth.schema.ts
│   ├── address.schema.ts
│   └── checkout.schema.ts
├── types/
│   └── auth.ts

```

```
|   ├── user.ts
|   ├── product.ts
|   ├── cart.ts
|   ├── inventory.ts
|   ├── order.ts
|   └── payment.ts
|
|   └── lib/
|       ├── utils.ts
|       ├── query-client.ts
|       └── constants.ts
|
|   └── providers/
|       ├── QueryProvider.tsx
|       └── AuthProvider.tsx
|
|   └── public/
|       ├── logo/
|       ├── icons/
|       └── placeholders/
|
|   ├── middleware.ts
|   ├── .env.example
|   ├── next.config.ts
|   ├── tailwind.config.ts
|   ├── tsconfig.json
|   ├── package.json
|   └── Dockerfile
```

33. Frontend Environment Variables

Only public/safe variables:

```
NEXT_PUBLIC_API_URL=
NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME=
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=
```

Never put these in frontend:

```
JWT_SECRET
DATABASE_URL
STRIPE_SECRET_KEY
CLOUDINARY_API_SECRET
RABBITMQ_URL
```

34. Frontend Implementation Order

1. Next.js + Tailwind Setup
↓
2. Design System
↓

3. Navbar + Footer
↓
4. Home Page
↓
5. Products + Categories
↓
6. Search + Basic Filters
↓
7. Product Details
↓
8. Login + Register
↓
9. Wishlist
↓
10. Cart
↓
11. Checkout + Address
↓
12. Stripe Payment UI
↓
13. Order Success
↓
14. My Orders + Tracking
↓
15. Profile
↓
16. Admin Layout
↓
17. Admin Dashboard
↓
18. Product CRUD
↓
19. Cloudinary Upload
↓
20. Categories
↓
21. Inventory
↓
22. Orders
↓
23. Payments / Refunds
↓
24. Basic Analytics
↓
25. API Integration
↓
26. Responsive Design
↓
27. Loading + Error States
↓
28. Frontend Testing

Initially:

Frontend
↓
Mock Data

After the backend is available:

Frontend
↓
services/*
↓
API Gateway
↓
Backend

Final Scope

It contains the **normal features a clothing shopping application needs**:

Customer: products, categories, search/filter, product details, size/color, authentication, wishlist, cart, checkout, address, Stripe payment, orders, tracking and profile.

Admin: dashboard, product CRUD, Cloudinary images, category CRUD, inventory, orders, payments/refunds and basic revenue/order analytics.

There is **no AI recommendation system, personalization engine, advanced analytics, multi-warehouse management, loyalty program, complex search engine, or other unnecessary premium functionality**.

With this plan u all(backend team) can create the implementation plan for the backend and once backend plan is created I will send u all that database and docker integration plan so read it carefully then make the backend plan